

ContextRay — White Paper Change Summary

v0.1.0 → 0.3.0

What changed between the two technical white papers: architecture, API contract, validation, verification, safety, limitations, and reference material. Built from the extracted text of the original v0.1 PDF and the current v0.3.0 codebase.

Property	v0.1.0 paper	0.3.0 paper
Version	0.1.0 (first public release)	0.3.0 (structural segmentation layer)
Pages	14	15
Pipeline stages	4 (chunk → detect → optimize → report)	5 + 4 (segmentation layer + core)
Public entry points	1 (optimize_context)	2 (optimize_context + optimize_text)
Return keys	4	5 (segments added)
Test status	191 checks — 4 API + 187 pipeline	222 checks — 18 API + 181 pipeline + 23 segmentation

Provenance: the v0.1.0 PDF was replaced in the v0.3 rebuild, but its complete text had been extracted first (14 pages, all sections) and is the source for the “Before” side throughout this document. “After” refers to the current ContextRay_White_Paper.pdf. Word-level rewording is trimmed; this document lists semantic changes only.

Contents

1 What Changed at a Glance	3
2 Section-by-Section Walkthrough	4
1 The Concept	4
2 Architecture	4
3 Public API Contract	4
4 Input Validation & Error Handling	4
5 Determinism & Verification	4
6 Empirical Findings (A/B with llama3.2:3b)	4
7 Safety Guarantees	4
8 Known Problems & Limitations	5
9 Integration Guide	5
10 Appendix A — Worked Example	5
11 Appendix B — Quick Reference	5
3 Brand-New Content	6
4 What Did NOT Change	7
5 Stale Data Corrected	8

1 What Changed at a Glance

Area	v0.1.0	0.3.0
Pipeline shape	Four stages; fences masked inside chunking; <code>optimize_context()</code> orchestrates.	Structural Segmentation stage runs first (<code>optimize_text()</code> path): STRICT segments masked then restored byte-for-byte; chunking fence masking kept.
Validation (Python API)	No documented Python-side error type; per-message checks existed but were CLI-documented.	Fail-fast <code>InvalidMessageError</code> (<code>ValueError</code> subclass) naming message index and exact key/type; 5-case table in Section 4.
Text input	CLI accepted a JSON message list only; anything else was rejected with “Invalid input format”.	CLI auto-detects: JSON list → <code>optimize_context()</code> ; any other text → <code>optimize_text()</code> ; <code>--strict</code> requires the JSON format.
Token estimates	Fixed <code>chars ÷ 4</code> heuristic, documented as approximate.	Heuristic stays; <code>token_estimator</code> callable replaces it everywhere; errors re-wrapped with context.
Failure behavior	Pipeline errors propagate; CLI catch-all prints one-line error.	Segmentation failure degrades to a safe plain-text fallback (<code>segmentation_fallback=true</code>) instead of failing.
Cost guards	Marker-length guard only (no negative reduction).	Plus: unfenced-Python scan firewall at 20,000 lines; CLI input size guard <code>--max-input-mb</code> (default 50 MB).
Verification	191 checks: <code>test_core.py</code> (4) + <code>test_pipeline.py</code> (187).	222 checks: + <code>token_estimator/error</code> cases, CLI text & JSON modes, SEGMENTS section, <code>--max-input-mb</code> ; new <code>test_segmentation.py</code> (23).
Reference material	11 sections; appendix covers chunking constants and CLI facts.	Same skeleton; added segmentation rules table, three new constants, and raw-text integration example.

2 Section-by-Section Walkthrough

1 The Concept

1.3 Design principles — previously three principles (byte-safety, determinism, conservative removal); now five. Added: **structural protection** (segmentation layer recognizes code/JSON structurally and masks it) and **fail fast, fail safe** (InvalidMessageError on bad input; documented plain-text fallback if segmentation fails). Core concept paragraphs unchanged.

2 Architecture

- Stages: 4 → **segmentation layer + 4-stage core** (5 modules). Diagram gained stage [0] “mask STRICT with length-preserving placeholders” and stage [4] “restore placeholders byte-for-byte”.
- **New 2.1 “Stage 0 — Structural Segmentation”**: detection-rule table (fenced code / fenced block / whole-line JSON / bare Python ≥ 3 lines / text), STRICT vs FLEXIBLE, placeholder padding, duplicate-STRICT collapse, 20,000-line firewall, fallback semantics.
- Old 2.1–2.4 renumbered to 2.2–2.5; chunking/detection/optimization text unchanged.
- 2.5 Reporting: added the “Segments:” section inserted into `optimize_text()` reports, and the token note naming which estimator was used.

3 Public API Contract

- 3.1: one entry point → **two**: `optimize_text(raw_text)` added; both expose stable signatures with `config + token_estimator` kwargs; a plain str is now also accepted by `optimize_context()`.
- Return contract: 4 keys → **5** — “segments” key (per-type counts/modes, `optimize_text` path only).
- metrics schema: 6 fields → **7** — `segmentation_fallback` added.
- New paragraph on `token_estimator`: replacement semantics, error wrapping, `est_tokens_saved = in – out`.
- 3.2 CLI: usage gained **--strict** and **--max-input-mb FLOAT**; input description “any text file”; auto-detection paragraph; “three of the five result keys”; terminal report layout now opens with the SEGMENTS block (text inputs).

4 Input Validation & Error Handling

- Before: CLI-only bullet list (file exists → valid JSON → top-level list → dict shape → string values) + catch-all handler.
- After: same CLI list, but the **size guard runs first** (before reading), and each shape failure now names the offending message index.
- New: Python API fail-fast section — InvalidMessageError case table (content None, content list, missing keys, non-dict items, non-string values).

5 Determinism & Verification

- 191 → **222 checks**: 18 API (validation errors, `token_estimator`) + 181 pipeline (CLI text/JSON modes, SEGMENTS section, `--max-input-mb`) + 23 new segmentation checks.
- `test_segmentation.py` (new): segment contiguity and byte-exactness, fence/JSON/bare-Python rules, STRICT survival, exact metrics with masking, precedence, line guard, fallback flag, segments breakdown.

6 Empirical Findings (A/B with llama3.2:3b)

No changes. The three findings, the reviewer table, and the guidance are reproduced verbatim — v0.3 introduced no new model-facing behavior.

7 Safety Guarantees

- Kept: system messages preserved, cross-role duplicates never removed, no negative reductions, tiny chunks conservative.

- Added: **STRICT segments survive byte-exact**; **placeholders never leak** (collision-free per run); **guarded worst-case costs** (20,000-line scan firewall + --max-input-mb); **safe fallback** (segmentation_fallback=true, never a crash).

8 Known Problems & Limitations

- “Code-fence coverage is narrow” rewritten as “Structural coverage is finite”: scope widened from fences only to fences + whole-line JSON + parseable bare Python (≥ 3 lines); inline code and malformed fences still chunk like text.
- Three rows added: unfenced-Python scan capped at 20,000 lines; fallback loses STRICT protection (flagged in metrics/report); CLI rejects files over --max-input-mb.
- Unchanged rows: exact bytes only, marker bias, whitespace-sensitive hashing, tiny chunks, cross-role duplicates, token heuristic, config ignored.
- Open questions updated: machine-readable role stats, configurable marker wording, MIN/MAX chunk size knobs.

9 Integration Guide

- Chat loop, agent flows, structured JSON content: unchanged.
- **New subsection “Raw text and documents”**: optimize_text() example with the segments key and byte-exact protection behavior.
- Golden rule now names both entry points (was: optimize_context is “the only documented public entry point”).

10 Appendix A — Worked Example

- sample_chat.json example and its report: unchanged (48 chars, 0%, tiny-chunk explanation).
- Realistic fixture report: gained the SEGMENTS block (code: 2 (protected), json: 1 (protected)) before IMPACT.

11 Appendix B — Quick Reference

- **New table “Segmentation decisions at a glance”** (5 detection rules → segment type/mode).
- Pipeline decisions table: unchanged.
- Constants: added SEGMENT_PLACEHOLDER, MAX_LINES_FOR_PYTHON_SCAN, MAX_INPUT_MB; the original five kept verbatim.
- CLI facts: added auto-detection/--strict bullet and the size-guard bullet.
- Project facts: version 0.1.0 → 0.3.0; tests 4+187 → 18+181+23; entry points 1 → 2.

3 Brand-New Content

New item	Where
segment_text() / Segment / mask_strict_segments() / restore_strict_segments()	2.1
segments result key and SEGMENTS report section	3.1, 3.2, 2.5
InvalidMessageError validation contract	4
segmentation_fallback metric and fallback semantics	3.1, 7, 8
token_estimator kwarg	3.1
--strict and --max-input-mb CLI flags; text auto-detection	3.2, 4
20,000-line unfenced-Python firewall	2.1, 7, 8, 11
Raw text & documents integration subsection	9
Segmentation decisions reference table	11

4 What Did NOT Change

- Core dedup semantics: global + per-role lookup tables, KEPT / REMOVED / FLAGGED_ONLY, same-speaker-only removal, system always kept.
- Chunk size constants (MAX_CHUNK_SIZE 1000, MIN_CHUNK_SIZE 64) and whitespace-folded sha256 hashing.
- Marker text and the negative-reduction guard (33-char marker, chunk kept when the marker is $\geq$ chunk length).
- Reporting arithmetic: waste = (count–1) × length, top 5 blocks, per-role stats, safety notes.
- Empirical A/B findings and their guidance (Section 6).
- Worked example input and its 0%-savings lesson (Appendix A).
- CLI exit codes (0 / 1 / 2), default output naming, --stdout/--output behavior, Unicode→ASCII marker fallback.
- Python $\geq$ 3.9, zero runtime dependencies, MIT license.

5 Stale Data Corrected

Item	Before (v0.1 paper)	After (v0.3 paper)
Version	0.1.0	0.3.0
Test status	191 checks — 4 API + 187 pipeline	222 checks — 18 API + 181 pipeline + 23 segmentation
Entry points	“only documented public entry point” (singular)	two documented entry points
Result keys written by the CLI	“three of the four result keys”	“three of the five result keys”
Limitation scope	fences only	fences + whole-line JSON + bare Python
Generated-from note	v0.1 codebase	v0.3 codebase

This change summary was generated from the extracted v0.1.0 PDF text and the v0.3.0 codebase. The current white paper can be regenerated at any time with `python scripts/build_white_paper.py`; this document with `python scripts/build_white_paper_diff.py`.